

Household Electricity Consumption Forecasting

Technical and Engineering Overview - companion source for a NotebookLM video.
Arturo Magdiel, 2026 - MIT License. Numbers are consistent with reports/report.pdf.

1. What this project is

This is a portfolio project that builds and rigorously evaluates a short-horizon forecaster for a single household's electricity demand, at two operational horizons: the next hour and the next day. It is engineered to production standards - public on GitHub, continuously tested, and deployed as an interactive dashboard - and documented for a technical audience rather than as an academic exercise.

It answers three business questions: first, can we reliably forecast next-hour and next-day consumption; second, what are the dominant temporal patterns (hourly, daily, weekly, seasonal); and third, which modeling paradigm offers the best performance-to-complexity trade-off. It also tests three hypotheses: H1, that weekly seasonality is statistically significant; H2, that the series is non-stationary in its raw form but stationary after seasonal differencing; and H3, that a tree-based model with lag features can match or outperform an LSTM on this univariate series.

2. The data

The dataset is the UCI Individual Household Electric Power Consumption series: 2,075,259 minute-level measurements from a home in Sceaux, France, spanning December 2006 to November 2010, with about 1.25 percent missing values, licensed CC BY 4.0. The raw minute data is aggregated to hourly resolution - the operationally relevant granularity, which also suppresses minute-level sensor noise - yielding 34,589 hourly rows measured in kilowatt-hours per hour. That hourly active-power series is the modeling target.

3. Repository and code architecture

The codebase is a modular Python package under `src/`, where each module has a single responsibility. `data_loader` handles hybrid acquisition: it loads the local file if present and verifies its SHA-256 hash, or downloads it through kagglehub and then verifies, so a stranger cloning the repository can obtain the data two ways. `data_quality` audits missing values, duplicates, outliers and physical range checks. `data_cleaning` imputes gaps - forward-fill for short gaps up to an hour, time interpolation for longer ones - and resamples to hourly. `feature_engineering` builds the leakage-safe feature matrix. `eda` and `stats_tests` produce the exploratory figures and the statistical tests. `modeling` holds the model roster, training and evaluation, while `metrics` is the single source of truth for every score, and `tracking` wraps MLflow. `timesfm_forecaster` adds the foundation-model benchmark. The end-to-end flow is: load, verify, audit, clean, resample, engineer features, split chronologically, train, cross-validate, tune, and evaluate.

4. Leakage-safe feature engineering

The forecasting matrix contains 22 features: calendar fields, cyclical sine and cosine encodings of hour, day-of-week and month, a French public-holiday flag, and causal lags of the target at 1, 24, 48, 168 and 336 hours, plus rolling mean and standard deviation over 24 and 168 hours, shifted so the current hour never enters its own window. Three exclusions are deliberate and important. The contemporaneous meter channels are dropped because they are unknown at forecast time, and one of them, the global current intensity, is a near-linear proxy of the target - including it would simply leak the answer. The calendar year is dropped because the training years, 2006 to 2009, are disjoint from the test year, 2010, and a tree cannot extrapolate an unseen value. Finally, all lag and rolling features are computed only after the chronological split. Designing features that use only information available at the moment of forecast is the backbone of a trustworthy time-series pipeline.

5. Modeling and validation

The split is strictly chronological: training runs from December 2006 to December 2009, which is 26,335 hourly rows, and testing runs from January to November 2010, which is 7,918 rows. It is never random, because a random split would place future observations in the training set and leak the very autocorrelation the models exploit. Model selection uses TimeSeriesSplit with an expanding window rather than K-fold. MAE is the primary metric, in the target's own kilowatt-hour units; MASE scales that error by the in-sample lag-24 seasonal-naive error on the training set - a denominator of 0.6551 - so a MASE below 1 means the model beats the simple rule of predicting the same hour yesterday.

The roster spans three paradigms of increasing complexity: three naive baselines (persistence, and daily and weekly seasonal naives), a SARIMA model with orders informed by the autocorrelation analysis, an XGBoost model on the lag features, and a two-layer LSTM in PyTorch. XGBoost was then tuned with a randomized search, but the gain was marginal, moving test MAE only from 0.3345 to 0.3338 - the headline being that on this series features, not hyperparameters, drive performance. A separate direct 24-hour-ahead XGBoost answers the next-day question.

6. Results

Among the trained models, tuned XGBoost is the winner: a next-hour MAE of 0.334 kilowatt-hours and MASE of 0.510, and a next-day MAE of 0.444 and MASE of 0.678. All three hypotheses are supported, including H3 - the gradient-boosted tree beats the LSTM in both cross-validation and on the test set, at a small fraction of the compute cost.

Model	Horizon	MAE (kWh)	MASE
xgboost_tuned (winner)	next-hour	0.334	0.510
sarima	next-hour	0.359	0.549
lstm	next-hour	0.371	0.567
naive_lag1 (persistence)	next-hour	0.395	0.603
xgboost_h24 (next-day)	next-day	0.444	0.678
timesfm 2.5 zero-shot *	next-hour	0.210	0.321

* TimesFM's lower error is not a genuine win - see Section 7.

7. The TimesFM foundation-model experiment

As an external benchmark, Google's TimesFM 2.5 - a 200-million-parameter pretrained time-series foundation model - was applied zero-shot, with no training on this series. Numerically it posted the lowest errors in the entire study, roughly 37 percent below tuned XGBoost at the next-hour horizon. Rather than take that at face value, the result was investigated, and it is almost certainly not a real win. The decisive tell is the horizon response: TimesFM's next-day error was essentially equal to, and even slightly below, its next-hour error, whereas a genuine causal forecaster degrades with horizon, as XGBoost does when it moves from 0.334 to 0.444. A next-day forecast that is as good as a next-hour forecast is behaving like recall, not extrapolation.

The likely cause is pretraining contamination: the UCI household-power series is a heavily mirrored public dataset that is very plausibly part of TimesFM's pretraining corpus, an advantage the trained models never had because they only ever saw 2006 to 2009. The custom forecaster itself was verified leakage-free - its context ends strictly before the forecast time and the target never enters the window - so the effect is upstream, in pretraining. Documented honestly, this is a benchmark curiosity with a contamination asterisk, not evidence that a general foundation model out-forecasts a task-specific tree on unseen data. Diagnosing it is the point: it demonstrates the kind of scientific skepticism that matters more than any single metric.

8. Software engineering and reproducibility

The project is engineered, not merely analyzed. It ships a 24-test pytest suite that runs on small synthetic fixtures, and a GitHub Actions workflow that runs those tests on every push without ever downloading the real dataset, keeping continuous integration fast, deterministic and independent of external APIs. Every source of randomness uses a fixed seed of 42, the environment is pinned to Python 3.12 with locked dependency versions, and experiments are tracked in MLflow. The heavy, optional TimesFM dependency is isolated in its own requirements file and imported lazily, so it never touches continuous integration or the deployment. Commits follow the Conventional Commits standard, so the git log itself tells the story of the project.

9. Dashboard, deployment and monitoring

A seven-page Streamlit and Plotly dashboard - Home, Exploration, Statistical Analysis, Model Comparison, Forecasting Playground, Scientific Report and Video - reads only the committed processed artifacts, so it runs anywhere without the raw dataset, and it is deployed on Railway straight from GitHub. The report also specifies a production monitoring strategy: data-drift detection on the input distributions, rolling MAE and MASE performance monitoring with an alarm when MASE approaches 1, and retraining triggers on drift, on a performance breach, or on a monthly cadence - which is cheap because the winning tree refits in seconds.

10. Narrative highlights for the video

The memorable beats are: a leakage-safe design where every feature respects what is knowable at forecast time; a reproduction of the well-known M4 and M5 competition result, where a gradient-boosted

tree beats a deep LSTM on a univariate series; a finding that hyperparameter tuning barely helped, because the feature engineering was doing the real work; a plot twist where a state-of-the-art foundation model appeared to win by a wide margin but was diagnosed as pretraining contamination; and a production-grade engineering wrapper of tests, continuous integration, an interactive dashboard and a documented monitoring plan.